

Problem no : 26153

Des:

- Background This challenge seeks AI systems capable of learning network behaviour, anticipating attacker progression and supporting proactive cyber defence using the emerging concept of World Models. Design and develop a software prototype that learns the evolving state of a computer network from traffic telemetry and predicts the likelihood and progression of malicious activity before compromise is completed. The solution should ingest network traffic, learn temporal behaviour, forecast future attack states and provide interpretable decision support for defenders. Solutions should demonstrate applicability to enterprise environments and Critical Information Infrastructure.

- Represent network state using feature vectors or graphs.
- Learn state-transition dynamics using sequence models (LSTM, Transformer), Graph Neural Networks, latent state models or other AI techniques.
- Forecast future network states and estimate the probability of attacker progression.
- Map predicted behaviour to recognised attack stages (e.g. MITRE ATT&CK).
- Provide explain ability using attention mechanisms, feature attribution or equivalent techniques •

Detailed Description Participants are encouraged to build world models based AI systems that move beyond static intrusion classification towards predictive cyber defence. The solution may utilise flow records, packet captures, authentication logs or other publicly available cybersecurity telemetry. It should model temporal relationships, infer evolving network state, predict future attack progression and present meaningful explanations for its predictions.

Traditional machine learning classifiers applied to network traffic treat each flow in isolation and map it to a binary benign/malicious label. This discards the temporal and causal structure of an infiltration: the sequence in which ports are probed, the pattern in which SYN flags precede ACK floods, the inter-arrival timing of reconnaissance packets before lateral movement begins. An infiltration is a process unfolding over time, not a single anomalous packet.

- World Models “ AI architectures that learn an internal causal simulation of how environment states evolve “ offer a fundamentally different approach. Rather than classifying traffic, a world model learns the transition dynamics $P(S_{t+1} | S_t)$: given the current observed network state (active flows, flag distributions, port activity, packet timing), what is the probability distribution over future states. This enables forward simulation: roll out K steps ahead and identify whether the current trajectory converges to an infiltration state, before the attacker completes the kill chain.

1. Input Data “ Two Levels of Traffic Feature Teams must work with both flow-level and packet-level features drawn from open-source network traffic datasets:

- Flow-level features (NetFlow / IPFIX format): source and destination IP/port pairs, TCP flag bitmask (SYN, ACK, FIN, RST, PSH, URG), protocol, bytes transferred per flow, packets per flow, flow duration, inter-arrival time (IAT) statistics (mean, variance, max), and bidirectional flow ratios.
- Packet-level features (PCAP-derived): Time-To-Live (TTL) values and their variance across a session, TCP window size, IP fragment flags, payload size distribution, port scan signatures (sequential or randomised port access patterns), and retransmission counts.

The combination of both levels is required because flow-level features capture aggregate behaviour (a SYN flood) while packet-level features expose timing and sequencing patterns (a slow reconnaissance scan designed to evade flow-based thresholds).

2. World Model Architecture The core deliverable is a learned model of network state transition dynamics â€” not a static classifier. The model must:

- Represent network state as a structured feature vector or graph encoding active flows at time t .
- Learn $P(S_{t+1} | S_t)$ â€” the probability distribution over the next network state given the current state â€” using a sequence model such as an LSTM, Temporal Transformer, or Graph Neural Network (GNN) operating over time-windowed traffic observations.
- Be trained on labelled open-source datasets using supervised dynamics learning, where ground-truth state transitions are derived from the attack timeline annotations in the dataset.
- Generalise to unseen attack patterns â€” not merely memorize signatures from the training set.

3. Infiltration Prediction and Attack Stage Mapping The world model must support forward simulation: given current observed traffic, roll out K steps and output

- A time-series probability score: likelihood of infiltration in the next K time windows.
- Predicted attack stage: mapping to MITRE ATT&CK phases â€” Reconnaissance, Initial Access, Lateral Movement, Command & Control, or Exfiltration â€” based on the predicted future state.
- Driving features: which specific flags, ports, or flow patterns are contributing most to the infiltration prediction (via attention weights or SHAP values).

The approaches are provided only as examples and are not mandatory. Teams are free to propose alternative architectures that satisfy the objectives.

- Expected Solution(Indicative)

A software-based, fully open-source solution is expected. The solution may include:

- A feature extraction pipeline that ingests CIC-IDS-2018 or CTU-13 CSV flow records and/or raw PCAP files (parsed using Scapy or PyShark) and outputs a timestamped, normalised feature matrix covering both flow-level and packet-level attributes described above.
- A trained world model (LSTM, Transformer, or GNN architecture) that demonstrably learns traffic state transition dynamics â€” not a static input-output classifier. Training scripts, model weights, and a reproducible training configuration must be included.
- An infiltration prediction engine that performs K -step forward simulation from a current traffic snapshot and outputs: infiltration probability score, predicted MITRE ATT&CK stage, and top contributing traffic features.
- An explainability output for each prediction â€” using SHAP values or model attention weights â€” identifying which flags, ports, or flow statistics are driving the prediction. Black-box outputs without interpretability are not acceptable.
- A working demonstration interface (Streamlit, Flask web app, or CLI) that accepts a PCAP or CSV file as input, runs the world model inference, and displays the infiltration probability timeline, flagged flows, and attack stage annotations. The interface must run fully offline without cloud API dependencies.
- Benchmark results comparing model performance (F1 score, precision, recall, false positive rate) against a logistic regression baseline trained on the same features, demonstrating that the world model's

temporal dynamics learning provides measurable improvement.

Dashboard Compulsory requirements:

Dashboard section	What	Required
1. Network Overview	Current network state, traffic volume, active flows, suspicious activity	Yes
2. Infiltration Probability	Probability of attack in next K time windows	Yes – core requirement
3. Attack Progression / Timeline	How the predicted attack evolves over time	Yes – core requirement
4. MITRE ATT&CK Stage	Predicted/current attack stage	Yes – core requirement
5. Explainability	Why the model made the prediction	Yes – core requirement
6. Flagged Traffic / Flows	Which IPs, ports, flags and flows are contributing	Yes

S.No	Requirement
1	Flow-level features (NetFlow/IPFIX: IPs/ports, TCP flags, bytes, packets, duration, IAT stats, bidirectional ratios)
2	Packet-level features (TTL + variance, TCP window size, fragment flags, payload size distribution, port-scan signatures, retransmission counts)
3	Network state as feature vector or graph
4	Learn $P(S_{t+1} S_t)$ via LSTM / Transformer / GNN / latent state model
5	Trained on labelled open datasets (CIC-IDS-2018 or CTU-13) with ground-truth transitions from attack timelines
6	Generalise to unseen attacks (not signature memorisation)
7	K-step forward simulation → infiltration probability time series
8	Map predicted state → MITRE ATT&CK stage

S.No	Requirement
9	Explainability via attention or SHAP, identifying driving features
10	Offline demo interface (Streamlit / Flask / CLI) accepting PCAP or CSV, showing probability timeline + flagged flows + stage annotations
11	Benchmark vs. logistic regression baseline (F1, precision, recall, FPR)
12	Reproducible training config + weights + scripts